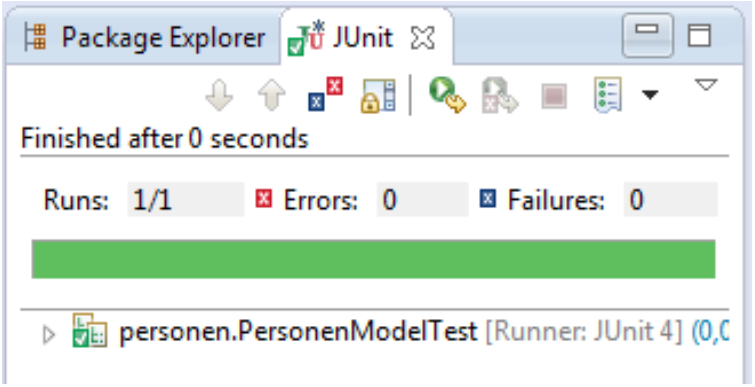
Hochschule Bochum
TECHNIK WIRTSCHAFT GESUNDHEIT

Objektorientierte Programmiertechniken



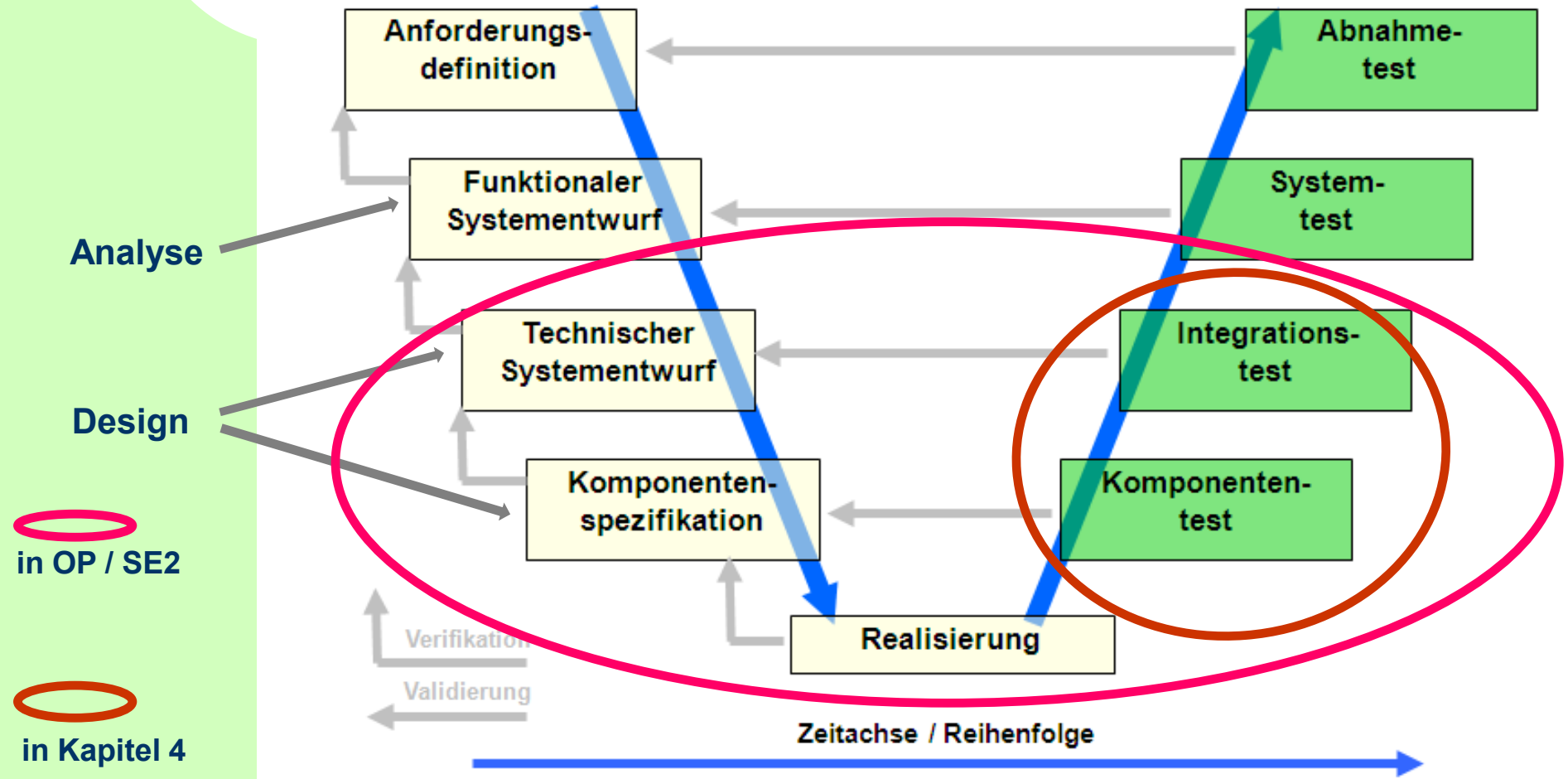
Prof. Dr. Ursula Oesing

Softwaretests

- Vorspann
- JUnit
- Ergänzungen
- Fazit

Vorspann

- Einordnung der Veranstaltung



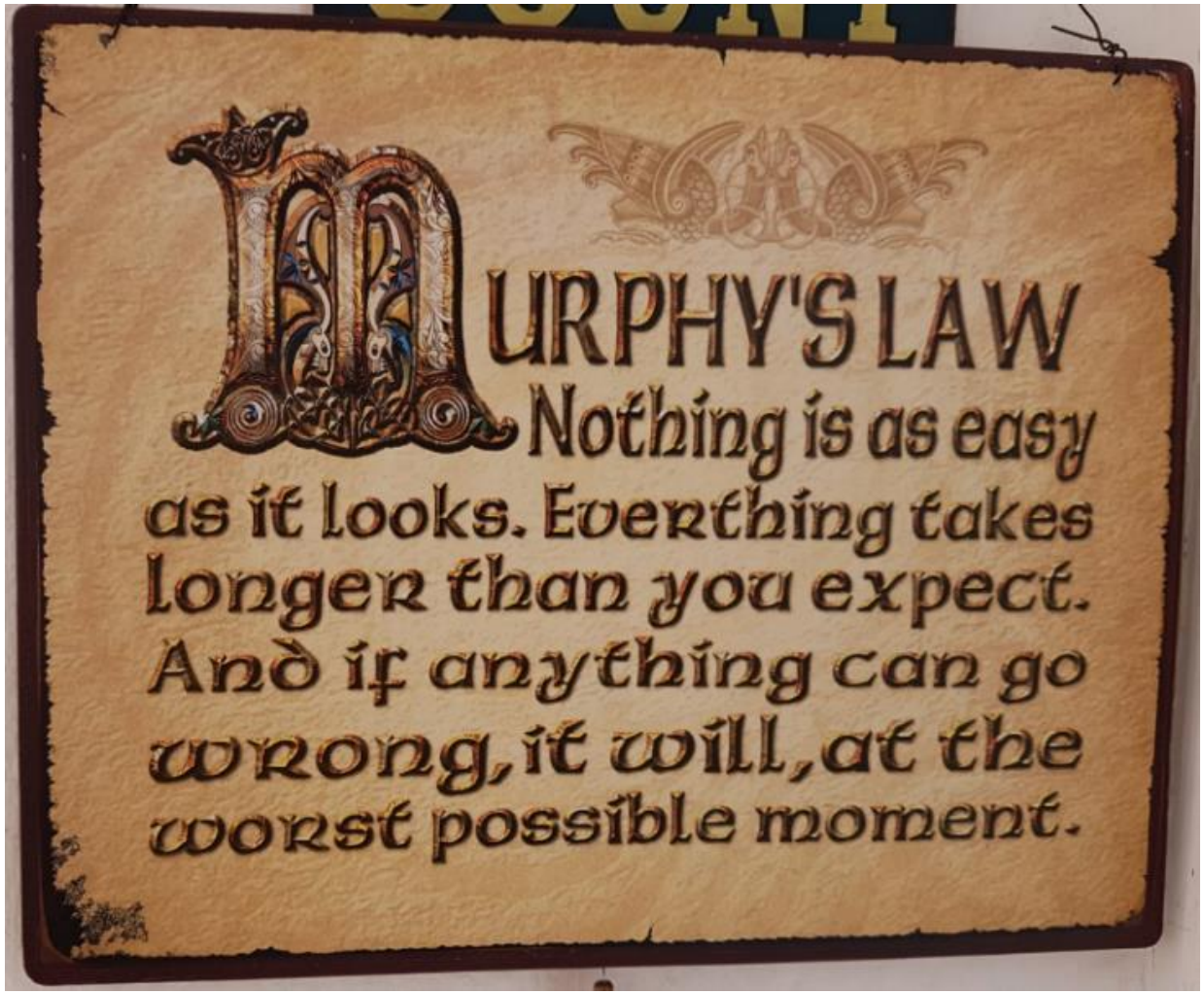
Vorspann

Vorspann

JUnit

Ergänzungen

Fazit

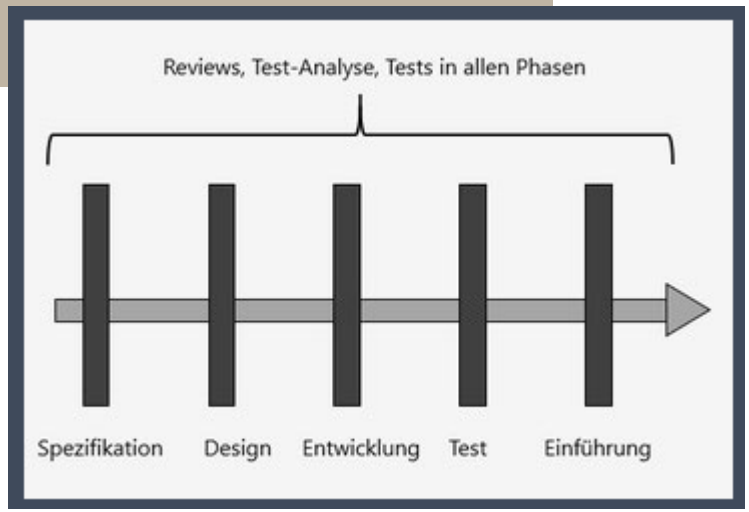


Vorspann

Wieso Softwaretests?

Ausgangssituation

Häufig beginnt das Testen erst mit Fertigstellung der Software und damit viel zu spät. Ein professionelles Software-Testing sollte bereits in der Analyse- und Designphase eines Projektes beginnen. Der Tester sollte Anforderungen und Fachkonzepte verifizieren und definiert damit die Qualitätskriterien.



https://www.dassachbuch.com/2020/05/24/ist-testen-von-software-wirklich-notwendig/

Vorspann
JUnit
Ergänzungen
Fazit

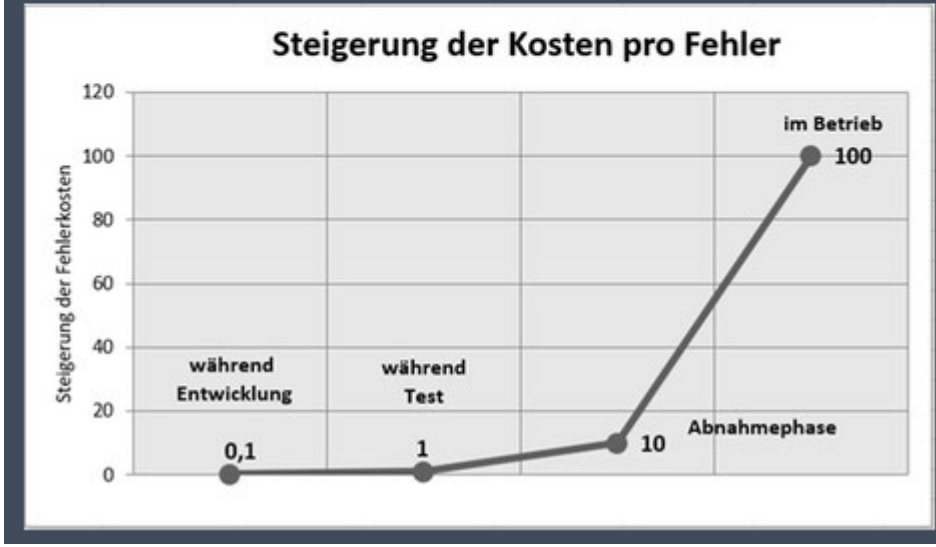
Vorspann

Wieso Softwaretests?

- Je früher Fehler gefunden werden, desto besser ist das für den Projektverlauf!
  -> Komponenten-tests und Integrationstests (mit JUnit)

Frühes Testen spart Zeit und Geld

Um Fehlerzustände früh zu finden, sollten sowohl statische als auch dynamische Testaktivitäten so früh wie möglich im Softwareentwicklungslebenszyklus gestartet werden. Frühes Testen im Softwareentwicklungslebenszyklus hilft dabei, kostenintensive Änderungen zu reduzieren oder vollständig zu vermeiden.



https://www.dassachbuch.com/2020/05/24/ist-testen-von-software-wirklich-notwendig/

Vorspann

JUnit

Ergänzungen

Fazit

Vorspann

- **Was ist teurer als Qualitätssicherung?**

Vorspann

JUnit

Ergänzungen

Fazit

JUnit

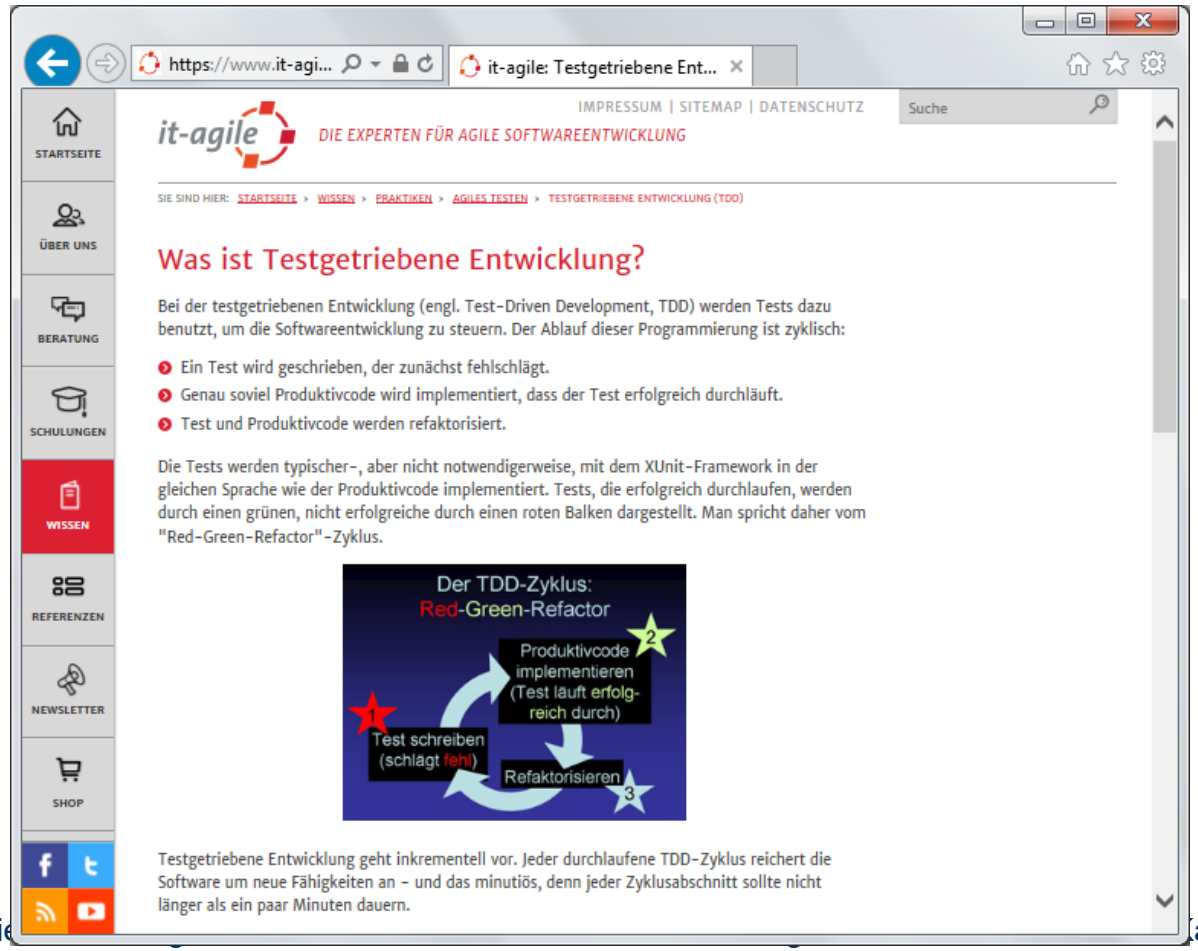
- JUnit Tests für Testgetriebene Entwicklung
- JUnit Tests in der Agilen Softwareentwicklung

Vorspann

JUnit

Ergänzungen

Fazit



JUnit

Vorspann

JUnit

Ergänzungen

Fazit

JUnit-Framework

- Automatisiertes Testen einzelner Units (z.B. Module, Klassen, Methoden) in Java
- Ist standardmäßig in Eclipse, NetBeans, IntelliJ … enthalten
- Wichtiges Hilfsmittel bei agilen Vorgehensweisen wie eXtreme Programming, Scrum:
  TDD: Zuerst werden die (JUnit-)Tests (Komponententests, Integrationstests) zum eigentlichen Quellcode programmiert, die den Quellcode testen. Diese werden noch nicht erfolgreich durchlaufen. Dann wird der Quellcode geschrieben, solange, bis die Tests erfolgreich durchlaufen.
- Tests sind automatisch wiederholbar:
  Falls bei der Weiterentwicklung Fehler eingebaut werden, werden diese durch die wiederholte Durchführung der Tests entdeckt.

JUnit

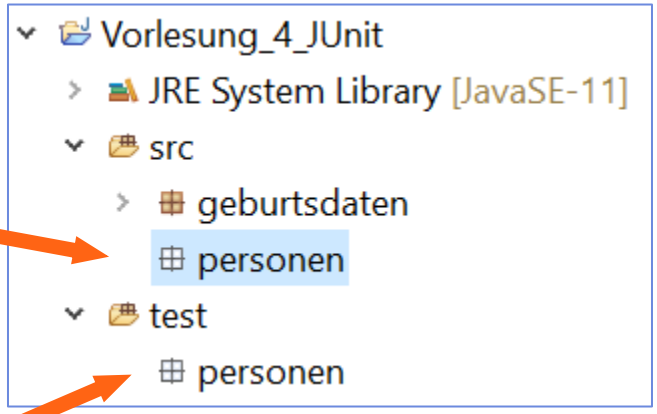
Vorspann

JUnit

Ergänzungen

Fazit

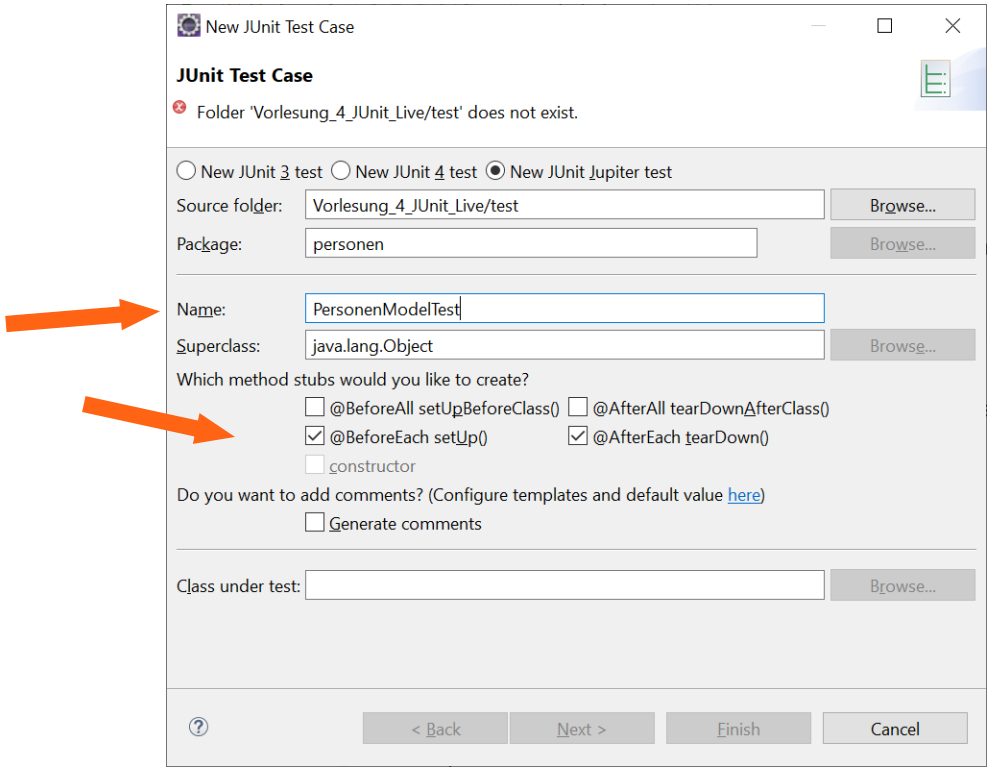
- **TDD (Test Driven Development)**
  - JUnit-Tests laufen nicht erfolgreich durch (, Tests werden **rot**), dann Korrekturen durchführen, bis alle Tests Ok sind (, Tests werden **grün**), dann Weiterentwicklung.
- **Beispiel: Einrichten eines JUnit Tests in Eclipse**
  - Anlegen von packages im Source-Folder src für den zu implementierenden Quellcode.
  - Anlegen eines Source-Folders test und Anlegen von packages für Testklassen im Source-Folder test wie im src-Folder src.



JUnit

Fortsetzung des Beispiels

- Sollen Methoden einer Klasse PersonenModel des src-Folders getestet werden, so legt man eine Klasse PersonenModelTest im test-Folder an.



Vorspann

JUnit

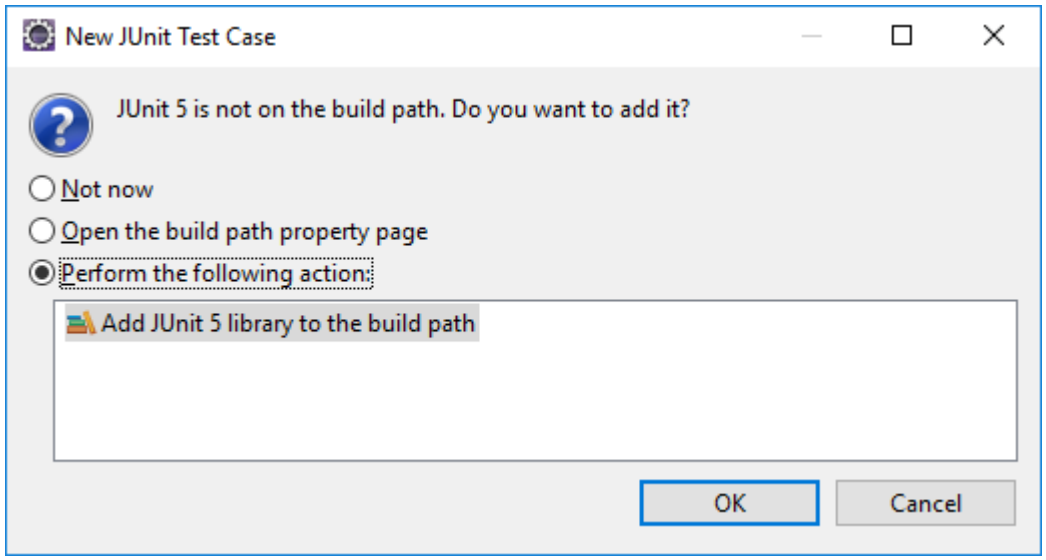
Ergänzungen

Fazit

JUnit

Fortsetzung des Beispiels

- Sollen Methoden einer Klasse PersonenModel des src-Folders getestet werden, so legt man eine Klasse PersonenModelTest im test-Folder an.

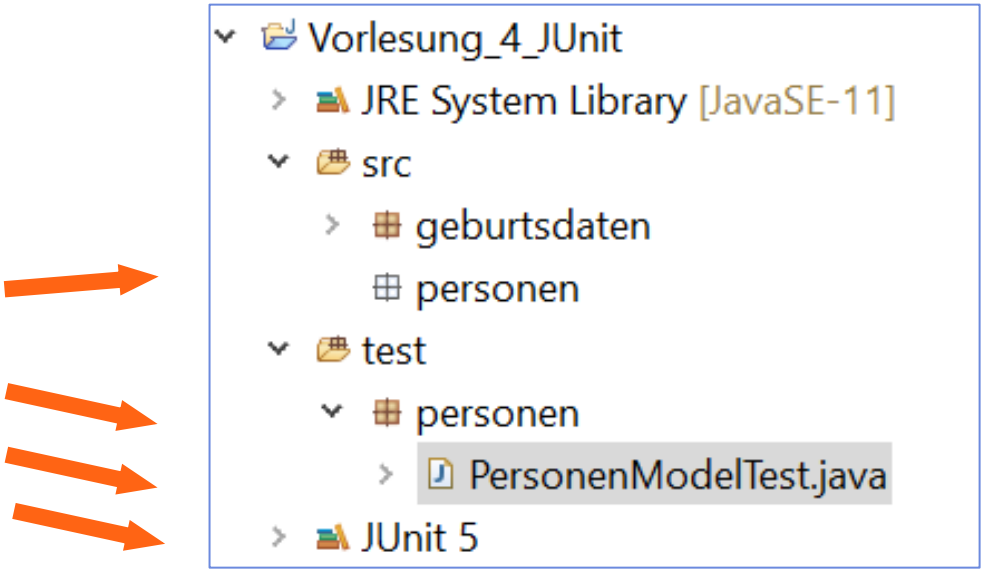


Vorspann
JUnit
Ergänzungen
Fazit

JUnit

Fortsetzung des Beispiels

- Sollen Methoden einer Klasse PersonenModel des src-Folders getestet werden, so legt man eine Klasse PersonenModelTest im test-Folder an.



Vorspann
JUnit
Ergänzungen
Fazit

JUnit

Vorspann

JUnit

Ergänzungen

Fazit

Fortsetzung des Beispiels

- Ergebnis:
  setUp() wird vor dem Durchlauf einer Testmethode, s.u., durchgeführt,
  tearDown() nach dem Durchlauf.
  Methoden, die mit @Test gekennzeichnet sind (Testmethoden), werden als Test durchgeführt.

```
package personen;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.*;

class PersonenModelTest {

    @BeforeEach
    public void setUp() throws Exception {
    }

    @AfterEach
    public void tearDown() throws Exception {
    }

    @Test
    public void test() {
        fail("Not yet implemented");
    }
}
```

JUnit

Vorspann

JUnit

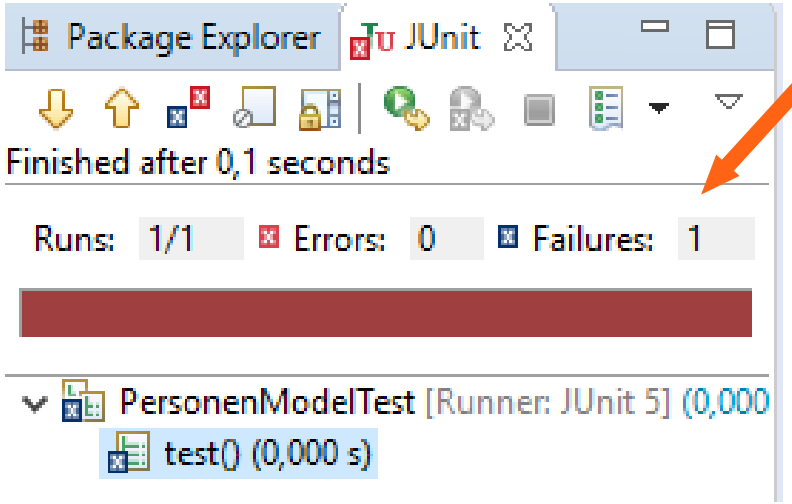
Ergänzungen

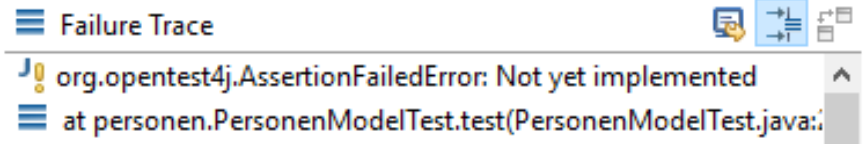
Fazit

Fortsetzung des Beispiels:

Run As / JUnit Test ergibt

- Ergebnis:
- fail ist eine statische Methode der Klasse Assertions. Wird diese aufgerufen, so erhält man ein Failure im JUnit-Test.
- Möglichkeit, Exceptions zu testen.





JUnit

Vorspann

JUnit

Ergänzungen

Fazit

Fortsetzung des Beispiels

- Zu testende Klasse PersonenModel wird erstellt (ohne Inhalt).
- Objekt der zu testenden Klasse wird in Personen-ModelTest deklariert.
- Mit dem (nicht initialisierten) Objekt wird eine Methode aufgerufen.

```
package personen;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.*;

public class PersonenModelTest {

    private PersonenModel personenModel;

    @BeforeEach
    public void setUp() throws Exception {
    }

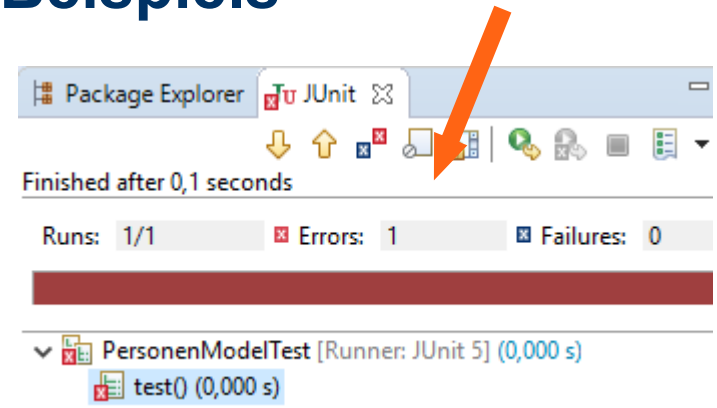
    @AfterEach
    public void tearDown() throws Exception {
    }

    @Test
    public void test() {
        this.personenModel.toString();
    }
}
```

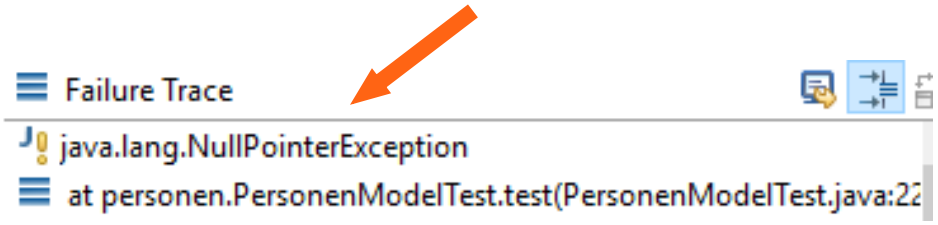
JUnit

Fortsetzung des Beispiels

- Ergebnis:
- Man erhält ein Error.



- Error:
  JUnit Test kann nicht durchlaufen.



- Failure:
  JUnit Test kann durchlaufen, aber der Test schlägt fehl:
  a) Fehler im JUnit Test oder
  b) Fehler im getesten Quellcode.

JUnit

Fortsetzung des Beispiels

- Klassen PersonenModel und Person

```
package personen;
public class PersonenModel {

    private Person[ ] personen = new Person[11];

    public Person[ ] getPersonen(){
        return this.personen;
    }
}
```

```
package personen;
public class Person{

    private String vorname;
    private String nachname;

    public Person(String vorname,
        String nachname) {
        this.vorname = vorname;
        this.nachname = nachname;
    }
    …
}
```

Vorspann
JUnit
Ergänzungen
Fazit

JUnit

Vorspann

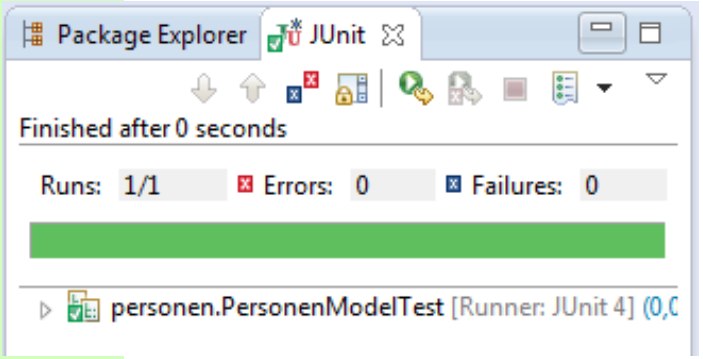
JUnit

Ergänzungen

Fazit

Fortsetzung des Beispiels

- Grüner Balken: JUnit Test ist erfolgreich durchgelaufen.




```
import static org.junit.jupiter.api
   .Assertions.assertTrue;
…
 public class PersonenModelTest {
    private PersonenModel   personenModel;

    @BeforeEach
    public void setUp() throws Exception {
        this.personenModel = new PersonenModel();
    }

    @AfterEach
    public void tearDown() throws Exception {
        this.personenModel = null;
    }

    @Test
    public void test() {
        assertTrue(personenModel.getPersonen()
            .length == 11,
            "Es wurden keine 11 Plätze angelegt.");
    }
 }
```

JUnit

Vorspann

JUnit

Ergänzungen

Fazit

Einschub: Annotationen

- Es können Metadaten in den Quelltext eingebaut werden.

Beispiele von Annotationstypen

- @Override
  Methode überschreibt Methode aus der Basisklasse.
  (Compilierfehler, falls das nicht der Fall ist.)
- @Deprecated
  Es wird abgeraten, dieses Element zu benutzen...

Beispiel

```
12
13      public static void main() {
14          int datum = new Date().getDate();
15      }
16  }
17
```

JUnit

Einschub: Annotation

- Eine Methode wurde auf @Deprecated gesetzt

Vorspann
JUnit
Ergänzungen
Fazit

```
@Test
void test() {
    // fail("Not yet implemented");
    // this.personenModel.toString();
    assertTrue(personenModel.getPersonen().length == 11,
        "Es wurden keine 11 P
}
```

The method getPersonen() from the type PersonenModel is deprecated

2 quick fixes available:

Add @SuppressWarnings 'deprecation' to 'test()'

Configure problem severity

PersonenModel... | Person.java | Geburtsdatum.... | Perso

```
1  package personen;
2
3  public class PersonenModel {
4
5      private Person[] personen = new Person[11];
6
7      @Deprecated
8      public Person[]  getPersonen(){
9          return this.personen;
10     }
11 }
```

JUnit

JUnit Test: Methoden aus der Klasse Assertions

- T1 sei eine Variable, die für einen Grunddatentyp oder ein Object steht.
  T2 sei eine Variable, die für double oder float steht.

Vorspann

JUnit

Ergänzungen

Fazit

Methode	Parameter	Funktionalität
assertArrayEquals	T1[] expecteds, T1[] actuals	vergleicht die Werte der vorgegebenen Arrays
assertArrayEquals	T2[] expecteds, T2[] actuals, T2 delta	vergleicht die Werte der vorgegebenen Arrays inkl. einer Toleranzabweichung
assertEquals	T1 expected, T1 actual	vergleicht die vorgegebenen Werte
assertEquals	T2 expected, T2 actual, T2 delta	vergleicht die vorgegebenen Werte inkl. einer Toleranzabweichung
assertEquals	T1 expected, T1 actual, String message	vergleicht die vorgegebenen Werte, ggf. Fehlermeldung
assertEquals	T2 expected, T2 actual, T2 delta, String message	vergleicht die vorgegebenen Werte inkl. Einer Toleranzab-weichung, ggf. Fehlermeldung

JUnit

JUnit Test: Methoden aus der Klasse Assertions

Methode	Parameter	Funktionalität
assertFalse	boolean condition	prüft, ob der vorgegebene Wert false ist
assertFalse	boolean condition, String message	prüft, ob der vorgegebene Wert false ist, ggf. Fehlermeldung
assertNotNull	Object object	prüft, ob die vorgegebene Variable nicht null ist
assertNotNull	Object object, String message	prüft, ob die vorgegebene Variable nicht null ist, ggf. Fehlermeldung
assertNull	Object object	prüft, ob die vorgegebene Variable null ist
assertNull	String message, Object object	prüft, ob die vorgegebene Variable null ist, ggf. Fehlermeldung
assertTrue	boolean condition	prüft, ob der vorgegebene Wert true ist
assertTrue	boolean condition, String message	prüft, ob der vorgegebene Wert true ist, ggf. Fehlermeldung

Vorspann
JUnit
Ergänzungen
Fazit

JUnit

Ergänzungen zu JUnit

- Überprüfung, ob Exceptions richtig geworfen werden, mit fail. Beispiel:

```
try {
    … // Handlung, die Exception auslöst
    fail("Exception muss geworfen werden, da …");
}
catch(Exception exc)
{ }     // Leerer catch-Block ist hier Ok.
```

- Anstatt eines leeren catch-Blocks könnte auch ein leerer finally-Block verwendet werden.
- Im Idealfall wird jede Zeile Quellcode durch einen JUnit-Testfall getestet werden!

Vorspann
JUnit
Ergänzungen
Fazit

JUnit

Vorspann

JUnit

Ergänzungen

Fazit

Aufgabe

- Schreiben Sie einen JUnit Test, welcher den Konstruktor der Klasse Person überprüft. Es soll getestet werden, ob die Attribute vorname und nachname tatsächlich mit den Parameterwerten gefüllt werden.
- Wie testet man, ob fehlerhafte Parameterwerte wie beispielsweise null behandelt werden?

```
package personen;
public class Person{

   private String vorname;
   private String nachname;

   public Person(String vorname,
         String nachname)  {
         this.vorname = vorname;
         this.nachname = nachname;
   }
   …
}
```

JUnit

JUnit und Java 5 /Java 8:
Methoden aus der Klasse Assertions

Vorspann

JUnit

Ergänzungen

Fazit

Methode	Parameter	Funktionalität
assertTrue	boolean condition, Supplier<String> messageSupplier	prüft, ob der vorgegebene Wert true ist
assertTrue	BooleanSupplier conditionSupplier, String message	prüft, ob der vorgegebene Wert true ist
assertTrue	BooleanSupplier conditionSupplier, Supplier<String> messageSupplier	prüft, ob der vorgegebene Wert true ist

JUnit

Vorspann

JUnit

Ergänzungen

Fazit

Aufgabe

- Schreiben Sie einen JUnit Test, welcher den Konstruktor der Klasse Person überprüft. Es soll getestet werden, ob das Attribut nachname tatsächlich mit dem entsprechenden Parameterwert gefüllt wird.
  Benutzen Sie
  a) void assertTrue(boolean condition,
  Supplier<String> messageSupplier)
  b) void assertTrue(BooleanSupplier conditionSupplier,
  String message)

JUnit

Vorspann

JUnit

Ergänzungen

Fazit

JUnit und Java 5 / Java 8: Methode aus der Klasse Assertions

<T> T assertThrows(Class<T> clazz, Executable executable)

- **Beispiel**





```
@Test
void test{
    Throwable exc
        = assertThrows(IllegalArgumentException.class,
        () -> {new Person("Elke", null);
            });
    assertEquals("Nachname darf nicht null sein!",
        exc.getMessage());
}
```

- Es wird überprüft, ob eine IllegalArgumentException geworfen wird.
- Es wird überprüft, ob die Fehlermeldung korrekt ist.

JUnit

Vorspann

JUnit

Ergänzungen

Fazit

JUnit und Java 5 / Java 8: Methode aus der Klasse Assertions

<T> T assertThrows(Class<T> clazz, Executable executable)

- **Ergänzung zu Executable**
  aus org.junit.jupiter.api.function.Executable
  (nicht aus java.lang.reflect)

  Executable ist ein Funktionales Interface.

```
public abstract interface org.junit.jupiter.api.function.Executable{
    public abstract void execute() throws java.lang.Throwable;
}
```

JUnit

Vorspann

JUnit

Ergänzungen

Fazit

JUnit-Motto:

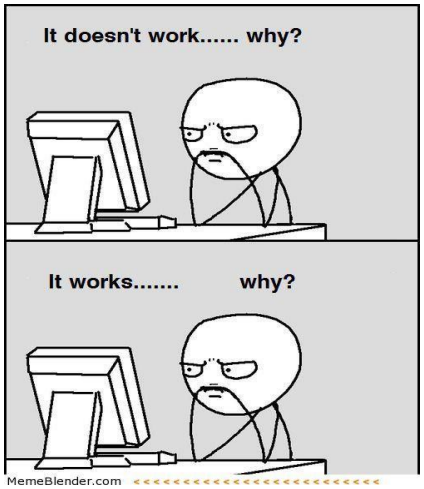
„ Keep the bar green to keep the code clean... “

Allgemeine Grundeinstellung:

„Das Programm mit dem Feature läuft ⇒ Es funktioniert.“

JUnit Grundeinstellung:

„Es gibt keinen automatischen Test für das Feature ⇒ Es funktioniert nicht.“



https://www.google.com/imgres?imgurl=https%3A%2F%2Fi.pinimg.com%2F550x%2F4f%2Fb5%2F88%2F4fb5886838c0492fc4b0cee3de87b648.jpg&imgrefurl=https%3A%2F%2Fwww.pinterest.de%2FJKhaveFun%2Fprogramming-memes%2F&tbnid=GCD3l5_jE9tiOM&vet=12ahUKEwjY2a7S1vL0AhWWxbsIHU50Di8QMygEegUIARCCAg..i&docid=TiEuG103ZIZYiM&w=500&h=641&q=programmieren%20lernen%20meme&hl=de&client=firefox-b-d&ved=2ahUKEwjY2a7S1vL0AhWWxbsIHU50Di8QMygEegUIARCCAg

Ergänzungen

Vorspann

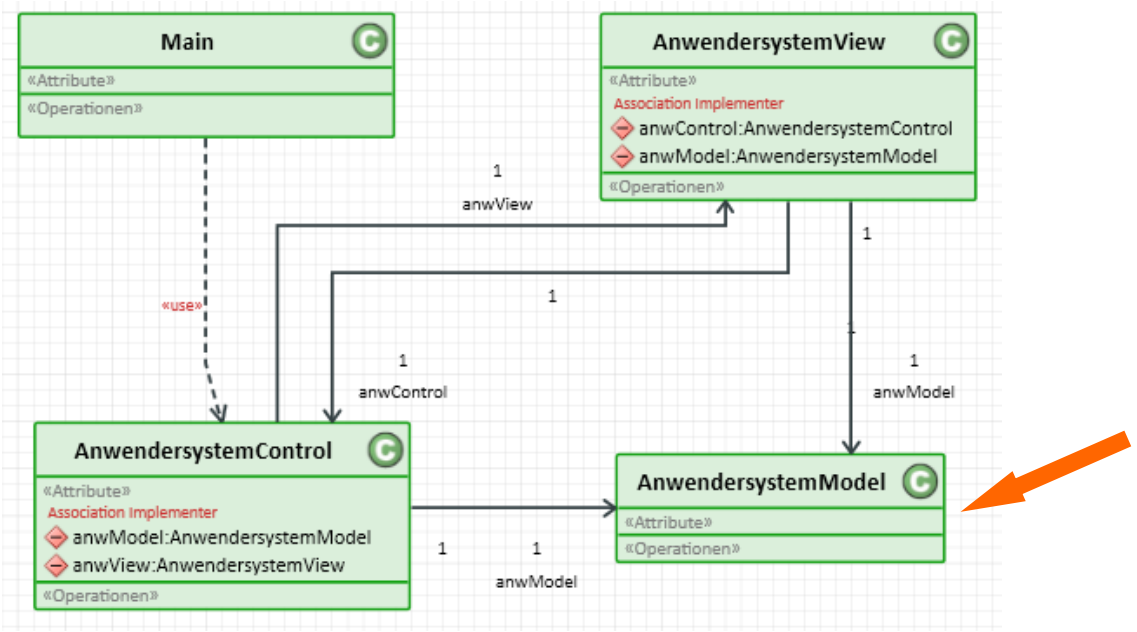
JUnit

Ergänzungen

Fazit

JUnit und MVC

- JUnit ist geeignet zum Test der Model-Klassen inklusive der Entity-Klassen



- JUnit ist **nicht** geeignet zum Test der View-Klassen!
- Wie ist die Situation, wenn MVC nicht verwendet wird?

Ergänzungen

Vorspann

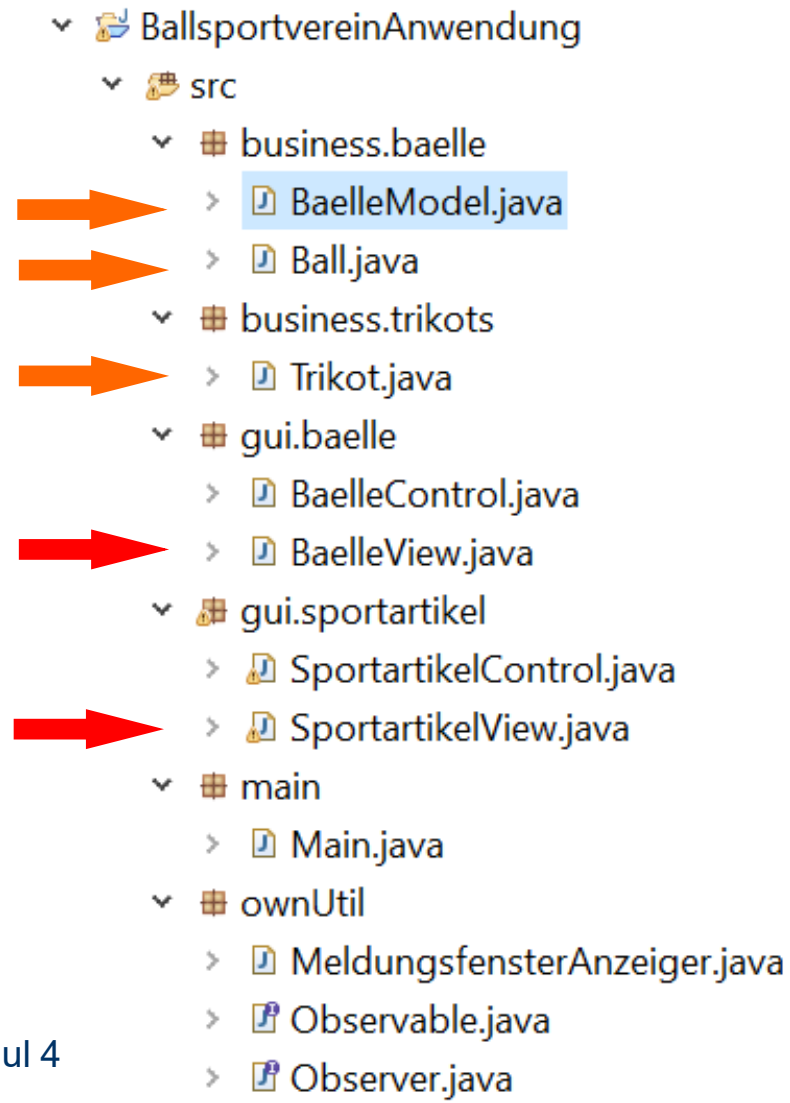
JUnit

Ergänzungen

Fazit

JUnit und MVC

- JUnit ist geeignet zum Test der Model-Klassen inklusive der Entity-Klassen.
- JUnit ist nicht geeignet zum Test der View-Klassen.



vgl. Übung Modul 4

Ergänzungen

Vorspann

JUnit

Ergänzungen

Fazit

Ergänzungen zu (Software-) Tests

- Nachholen aus SE:
- **Im WiSe 2025/2026 nachholen: SoftwareEngineering/Vorlesung/SE_4.pptx ->Qualitätssicherung ab Folie 22, ggf. Plausis auslassen, falls zu wenig Zeit**
- **Nachholen:**
  Software Engineering/Vorlesung/SEoderOP/SE_OP_4.pptx und
  SE_OP_AufgabeZurKlausur.docx und
  Software Engineering/Uebung/SEoderOP /
  UebungModul_Aufgaben_ZustandsbasierterTest_Entscheidungstabellen.docx

  -> Äquivalenzklassentest
  -> Grenzwertbetrachtung
  -> Use Case Test
  -> Zustandsbasierter Softwaretest
  -> Entscheidungstabellen
  -> Performancetests mittels VisualVM

Fazit

Vorspann

JUnit

Ergänzungen

Fazit

Wieso Komponententests und Integrationstests mit JUnit?

- JUnit ist für die Durchführung und Automatisierung von Komponententests und Integrationstests geeignet.
- auch für andere Sprachen: CUnit, PyUnit, …

- Unit Tests bilden die Brücke zwischen Fachlichkeit und Programmierung.



Claude Monet:
Seerosenteich Japanische Brücke

Fazit

Vorspann

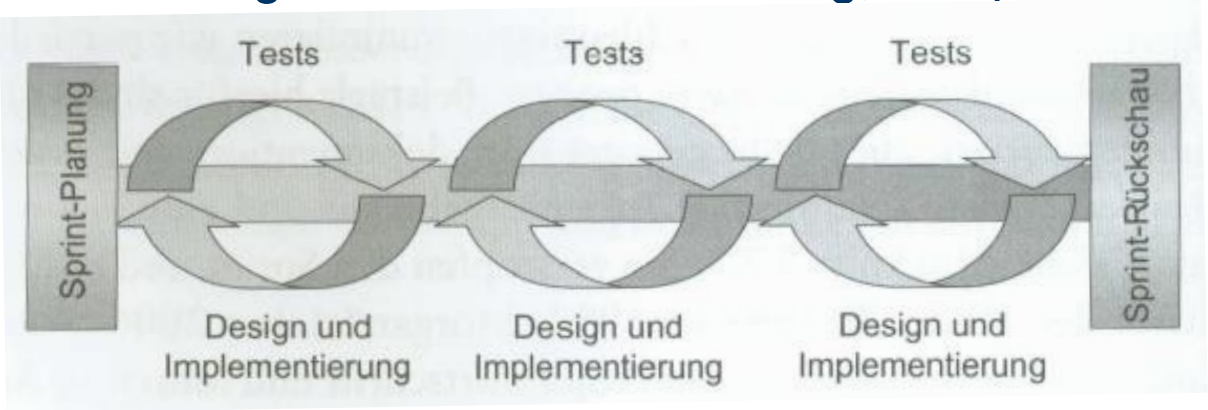
JUnit

Ergänzungen

Fazit

➢ Wieso sollten Sie Komponententests und Integrationstests kennen?

- Für TDD bei Agiler Softwareentwicklung, Beispiel Scrum:



Roman Pichler: Scrum, Agiles Projektmanagement erfolgreich einsetzen

- Beispiel Scrum: Interdisziplinär besetzte Teams

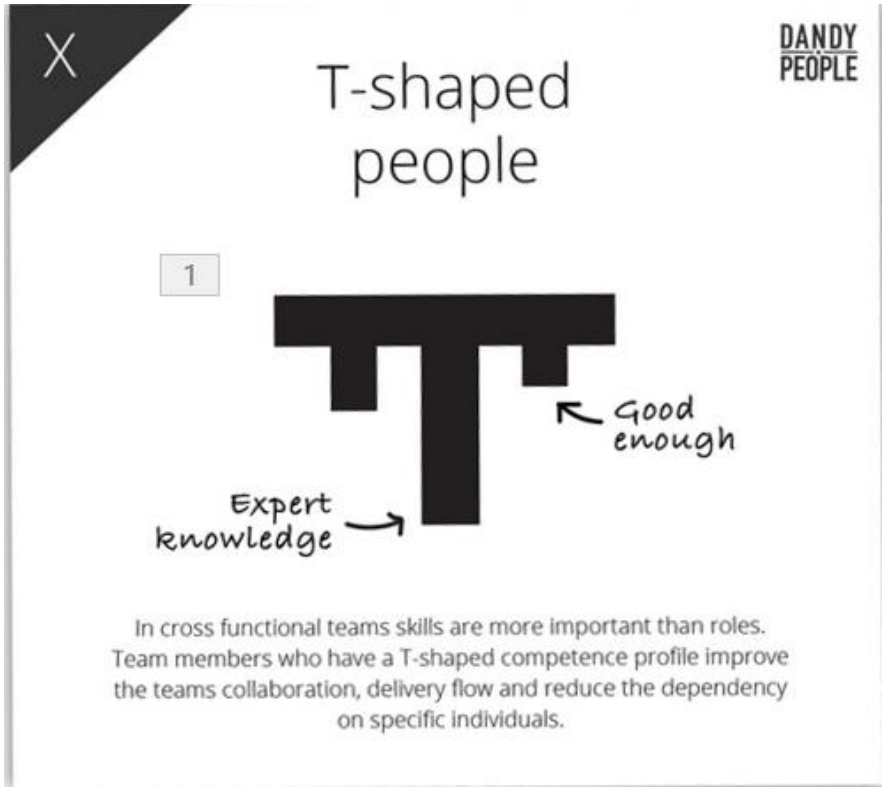
aus https://scrum-master.de/Scrum-Rollen/Scrum-Rollen_Team

Merkmale und Eigenschaften eines Scrum-Teams

- besteht aus fünf bis zehn Personen (ideal sind sieben)
- größere Gruppen werden in mehrere unabhängige, aber miteinander kommunizierende Teams aufgeteilt
- ist interdisziplinär zusammengesetzt (Entwickler, Architekten, Tester, technische Redakteure), wenn von Vorteil - und das ist meistens der Fall

Fazit

- **Wieso sollten Sie Komponententests und Integrationstests kennen?**
  - Beispiel Scrum: T-Shape aus https://dandypeople.com



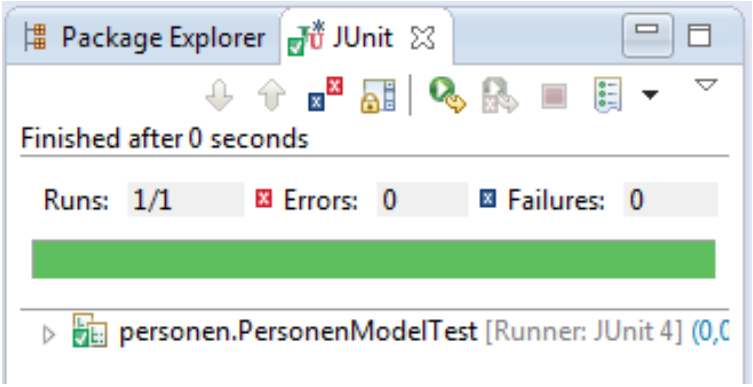
Vorspann
JUnit
Ergänzungen
Fazit

Softwaretests

- Vorspann ✓
- JUnit ✓
- Ergänzungen ✓
- Fazit ✓

Hochschule Bochum
TECHNIK WIRTSCHAFT GESUNDHEIT

Objektorientierte Programmiertechniken



Vielen Dank für Ihr Interesse!